
COMPUTER SCIENCE MENTORS 61A

September 23 – September 26, 2025

1 Recursion

1. Complete the definition for `num_digits`, which takes in a number n and returns the number of digits it has. Implement the function recursively.

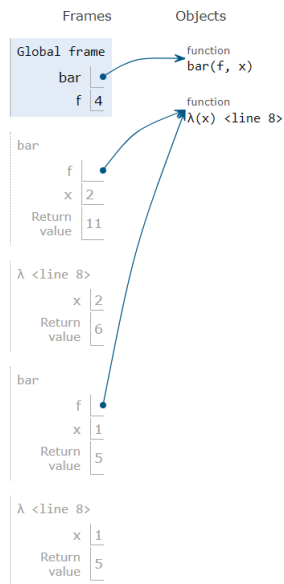
```
def num_digits(n):  
    """Takes in an positive integer and returns the number of  
    digits.  
  
    >>> num_digits(0)  
    1  
    >>> num_digits(1)  
    1  
    >>> num_digits(7)  
    1  
    >>> num_digits(1093)  
    4  
    ""  
  
    if n < 10:  
        return 1  
    else:  
        return 1 + num_digits(n // 10)
```

2. Draw the environment diagram that results from running the code.

```
def bar(f, x):  
    if x == 1:  
        return f(x)  
    else:  
        return f(x) + bar(f, x - 1)
```

```
f = 4  
bar(lambda x: x + f, 2)
```

Output: 3



<https://goo.gl/kR2n9M>

3. Fill in `collapse`, which takes in a non-negative integer `n` and returns the number resulting from removing consecutive repeating digits, i.e. while the number may have digits repeating elsewhere in the number, the same number cannot appear next to itself.

```
def collapse(n):  
    """  
    >>> collapse(9)  
    9  
    >>> collapse(12)  
    12  
    >>> collapse(22)  
    2  
    >>> collapse(12234441)  
    12341  
    >>> collapse(11200000013333)  
    12013  
    """  
    rest, last = n // 10, n % 10  
  
    if _____:  
        _____  
  
    elif _____:  
        _____  
  
    else:  
        _____
```

```
def collapse(n):  
    rest, last = n // 10, n % 10  
    if rest == 0:  
        return last  
    elif last == rest % 10:  
        return collapse(rest)  
    else:  
        return collapse(rest) * 10 + last
```

2 Tree Recursion

1. The *Gibonacci sequence* is a recursively defined sequence of integers; we denote the n th Gibonacci number g_n . The first three terms of the sequence are $g_0 = 0, g_1 = 1, g_2 = 2$. For $n \geq 3$, g_n is defined as the sum of the previous three terms in the sequence.

Complete the function `gib`, which takes in an integer `n` and returns the n th Gibonacci number, g_n . Also, identify the three parts of recursive function design as they are used in your solution.

```
def gib(n):  
    """  
    >>> gib(0)  
    0  
    >>> gib(1)  
    1  
    >>> gib(2)  
    2  
    >>> gib(3) # gib(2) + gib(1) + gib(0) = 3  
    3  
    >>> gib(4) # gib(3) + gib(2) + gib(1) = 6  
    6  
    """  
    if _____:  
          
        return _____  
      
    return _____
```

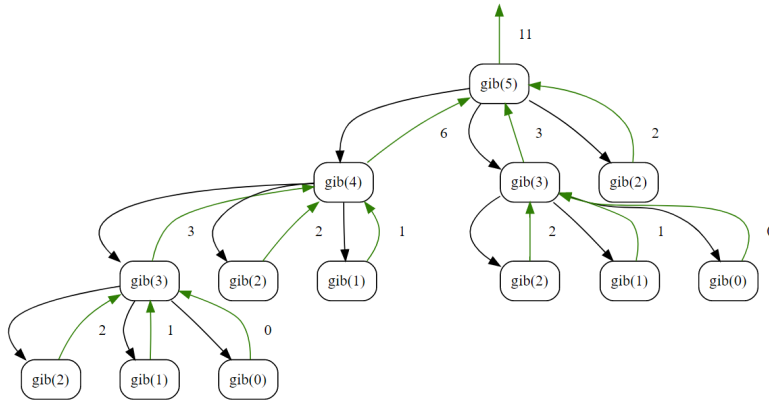
```
def gib(n):  
    if n <= 2:  
        return n  
    return gib(n - 1) + gib(n - 2) + gib(n - 3)
```

- Base case: `if n <= 2: return n`
- Recursive calls: `gib(n - 1), gib(n - 2), gib(n - 3)`
- Solving the larger problem: adding the results of the three recursive calls.

2. Including the original call, how many calls are made to `gib` when you evaluate `gib(5)`?

13.

There are a few ways you could determine this. First, you could draw out a call graph that shows all of the calls that are made. <https://tinyurl.com/gibsol>



You could also build things from the ground up using recursive intuition.

- `gib(0)` takes one call because it's a base case.
- `gib(1)` takes one call because it's a base case.
- `gib(2)` takes one call because it's a base case.
- `gib(3)` takes four calls: one for the original call, one for `gib(0)`, one for `gib(1)`, and one for `gib(2)`.
- `gib(4)` takes seven calls: one for the original call, four for `gib(3)`, one for `gib(2)`, and one for `gib(1)`.
- `gib(5)` takes 13 calls: one for the original call, seven for `gib(4)`, four for `gib(3)`, and one for `gib(2)`.

This second method is much faster, especially for higher values of `n`.

3. Gabe's Donut shop has an unlimited supply of f different flavors of donuts. Adit wants to buy a box containing d donuts. Complete the skeleton for the function `donut`, which determines the number of possible ways there are for Adit to select his d donuts from the f flavors. You may assume that d and f are non-negative integers.

Hint: Does order matter?

```
def donut(d, f):  
    """  
    >>> donut(12, 1)  
    1  
    >>> donut(12, 2)  
    13  
    >>> donut(12, 12)  
    1352078  
    >>> donut(0, 0)  
    1  
    """  
    if _____:  
        return _____  
  
    if _____:  
        return _____  
  
    return _____  
  
def donut(d, f):  
    if d == 0:  
        return 1  
    if f == 0:  
        return 0  
    return donut(d - 1, f) + donut(d, f - 1)
```

Suppose the flavors are numbered 1 through f . Because order does not matter, we're going to say that Adit fills up his donut box starting with flavor number f and working down to flavor number 1. Under this scheme, Adit has a choice to make: does he want a donut of flavor number f , or does he not want one? If he does initially take a donut of flavor number f , then the number of ways to select the remaining $d - 1$ slots of the box is `donut(d - 1, f)`. If he does not want a donut of flavor number f , then he has d slots left in the box and $f - 1$ flavors to fill them with, making the number of ways to make the selection `donut(d, f - 1)`. Since he must make the choice one way or the other, the total number of ways to fill up the box is `donut(d - 1, f) + donut(d, f - 1)`.

4. Imagine you are taking the 61A midterm. There are n questions left on the exam and t time remaining. Completing question i takes i time but gives you $i + 50$ points on the exam. Write a program that will output the highest possible score on the exam given n and t (note: you can't take more than t time).

```
def midterm(n, t):
    """
    >>> midterm(500, 0) # No time left!
    0
    >>> midterm(3, 3) # 51 + 52, questions 1 & 2
    103
    >>> midterm(3, 5) # 52 + 53, questions 2 & 3
    105
    >>> midterm(4, 9) # 52 + 53 + 54, questions 2 & 3 & 4
    159
    """
    if _____:

        return _____

    else:

        return _____

    if n <= 0 or t <= 0:
        return 0
    else:
        return max(n + 50 + midterm(n - 1, t - n), midterm(n - 1, t))
```